HKPUG Meetup | June 2026

Building a Spatial Vision Framework Completely Alone Handcrafting L.E.P.A.U.T.E. Geometry & Asynchronous Code

A solo developer's journey hacking simulators, pipeline architecture, and multithreading locks

Speaker: Carson Wu
CV & NLP Solo Developer

Hong Kong Python User Group
Personal Project Presentation

Why did I start this? (I hate brute-forcing data)

Most autonomous vision frameworks are massively heavy. As a solo dev, I don't have a massive cluster to train giant models on millions of miles, so I thought: **Can we use geometric laws to force the network to take a shortcut?**

My core design principles to make this possible:

- **Physics instead of memory:** Map continuous sequence frames directly onto $SE(3)$ Lie groups, so spatial awareness is baked into the math.
- **Enforcing consistency:** Ensure that when the vehicle pitches or rolls in the simulator, the extracted features don't lose their spatial orientation.

- **Stitching new tools together:** Along with optical flow, I threw in Transformer structural embeddings and a Zero-shot Open-Vocabulary classifier to see how far I could push it.

Solo Architecture: Keeping things alive by breaking threads

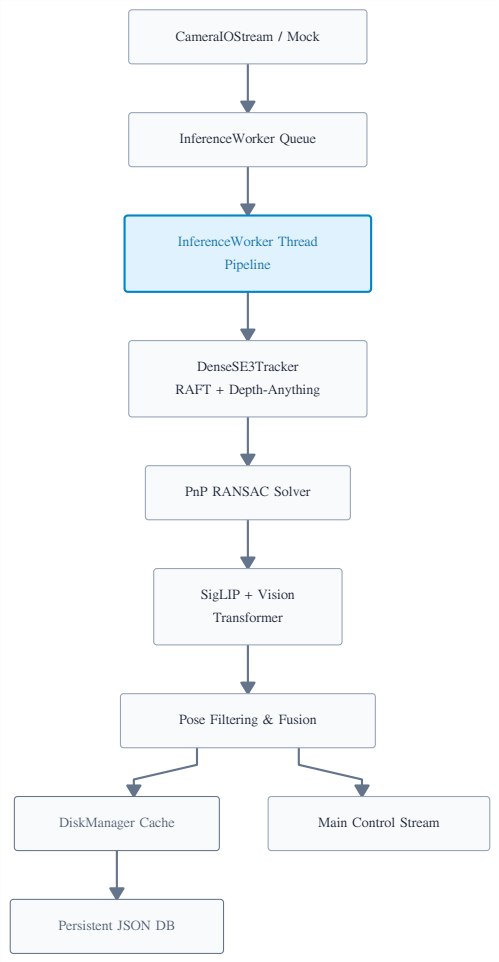
Full disclosure: **This framework hasn't touched a real vehicle yet.** But testing on my desktop simulator setup taught me early on that mixing ingestion with inference breaks everything immediately.

- **Isolated Ingestion Ring:** Image capture runs on its own isolated loop. No matter how much the downstream models lag, the camera/simulator stream never drops a frame.
- **Asynchronous Inference Pool:** Heavyweights like Depth-Anything and SigLIP are thrown into separate worker threads to crunch data at their own pace.

- **Zero-Overhead Persistence:** Atomic operations dump telemetry directly into caches and local JSON files so I can review exactly where my tracking drifted post-run.

How the Pipeline Loops in the Background

This is the asynchronous topology running locally on my development machine:


CameraIOStream / Mock
InferenceWorker Queue
InferenceWorker Thread
Pipeline
DenseSE3Tracker
RAFT + Depth-Anything
PnP RANSAC Solver
SigLIP + Vision
Transformer
Pose Filtering & Fusion
DiskManager Cache
Main Control Stream
Persistent JSON DB

Hardcore Geometry: Why bother with SE(3)?

Without geometric constraints, if your vehicle hits a massive bump in a virtual world and shakes violently, the model completely loses its bearings. I wanted my code to have an innate “sense of balance.”

Lie Algebra Spatial Mapping:

Sequential updates can be translated directly into explicit Lie vectors:

$$\xi = (v, \omega) \in \mathfrak{se}(3)$$

When a movement warps the input sequence by some transformation $g \in SE(3)$, the feature extractor $f(x)$ naturally flows with it:

$$f(g \cdot x) = g \cdot f(x)$$

- **The real-world benefit:** Forcing this mathematical constraint straight into the tensor layers gives the system geometric memory, keeping tracking stable even during erratic movement.

The Joy of Custom Code: Differentiable Warping & Rings

1. Blind Track Tracking

- I wrote custom geometric differentiable layers to warp feature maps dynamically based on estimated physical transforms.
- Even without any simulated GPS or pre-mapped environments, the system extracts a clean 6-DoF trajectory purely out of pixel changes.

2. Squeezing Performance with Ring Buffers

- Running on a single GPU means inference speeds struggle to match raw video framerates.
- I built a Thread Ring protected by atomic parameter locks, discarding old frames seamlessly so the control loop only reads the latest state.

Open-Vocabulary and Multi-Tasking (Where I lost my hair)

Handling Unexpected Obstacles

- I plugged SigLIP's zero-shot classification embeddings straight into the perception pipeline.
- If an odd object—like a strange cardboard box or fallen debris—appears in the simulator, I don't have to retrain. I just pass a new label at runtime to detect it.

The Balancing Act of Multi-Loss

- This was my absolute biggest pain point: ego-pose tracking requires Huber-loss regressions, but visual parsing needs $NT - Xent$ contrastive loss.
- Balancing these two entirely different objectives alone felt like walking a tightrope—but after countless nights, they finally converged together.

Defensive Programming: Handling Failures at Home

- **Sim/Sensor Disconnect Recovery:** If a webcam drops or the simulator's webhook hits a lag spike, the ingestion layer automatically flips to a mathematical parametric generator. It extrapolates frames using the last known velocity to keep the system from throwing an exception and crashing.
- **Hard Drive Protection:** Long testing sessions create massive amounts of telemetry. The `DiskManager` acts as a circular cache, holding onto the last 2,000 files and overwriting the rest so I don't wake up to a filled hard drive.

What This Taught Me in Virtual Worlds

Even though this code hasn't seen a single mile of asphalt yet, the results on my testbench show massive potential for solo developers:

- **Math scales better than data:** It proves you don't need a massive team throwing datasets at a wall. Strict geometric priors allow lightweight setups to excel in spatial mapping.
- **Isolation brings stability:** Separating the threads makes the pipeline incredibly robust against latency spikes and hardware stuttering.
- **Dynamic adaptability:** Open-vocabulary setups make it trivial to introduce new object classes on the fly without running heavy training loops.

How to Run It: Runtime Configurations

When deploying this on my local PC or a standalone testboard, I use simple environment flags to keep configuration overhead to zero:

```
export LEPAUTE_DEVICE="cuda"
export LEPAUTE_MAX_DISK_FILES=2000
export LEPAUTE_MODE="autonomous"
```

If you want to look at the raw matrix values spitting out of the geometric layers in real time, simply drop the logging level down to Debug.

Code Execution: Three Lines to Fire Up the Engine

The underlying multithreading code and C++ geometric wrappers took months to figure out, but the final Python API is dead simple:

```
import logging
from lepaute.core import CameraIOStream, InferenceWorker

logging.basicConfig(level=logging.DEBUG)

stream = CameraIOStream(device_index=0, mode="autonomous")

worker = InferenceWorker(source=stream)
worker.start()
```

Q&A

Thank You!